

INTERNSHIP TRAINING REPORT

Quick Bites: A Food Delivery and Restaurant Aggregator

A Training Report

Submitted in partial fulfillment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science and Engineering

Submitted to

LOVELY PROFESSIONAL UNIVERSITY

PHAGWARA, PUNJAB

Duration of Training: October 22, 2025 to April 14, 2026



SUBMITTED BY:

Name: Suryazi Mustafeezulla Khan

Reg. No: 12201534

Branch: CSE

SUBMITTED TO:

Name: Dr. Amit Kumar

Designation: Assistant Professor

School of CSE, LPU

STUDENT DECLARATION

I, **Suryazi Mustafeezulla Khan**, hereby declare that this internship training report is based on my practical learning and experience gained during the analyst training at **Capgemini Technology Services India Limited**. The report has been prepared by me and contains original work, observations, and knowledge gained during the training period.

The report includes details about the technologies learned, project work, practical implementation, and software development concepts understood during the training program. This report is submitted for the partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**.

Suryazi Mustafeezulla Khan

Registration Number: 12201534

Signature of the Student

DECLARATION BY THE SUPERVISORS

TO WHOM SO EVER IT MAY CONCERN

This is to certify that **Suryazi Mustafeezulla khan**, Registration Number **122101534** from Lovely Professional University, Phagwara, Punjab, has worked as a trainee in **Capgemini Technology Services India Limited** on the project title "**QuickBites : An Enterprise-Grade Food Delivery and Restaurant Aggregator**" under my supervision from **October 17, 2025 to May 14, 2026**. It is further stated that the work carried out by the student is a record of original work to the best of my knowledge and belief for the partial fulfillment of the requirements for the award of the degree, Bachelor of Technology in Computer Science and Engineering.

Puneet Kumra

External Supervisor

Head - Fresher Hiring

Capgemini Technology Services

Dated: June 3, 2026

Dr. Amit Kumar

Internal Supervisor

Assistant Professor

Lovely Professional University

INTERNSHIP LETTER & LOI

The candidate has successfully completed her industrial training program as an Analyst at Capgemini Technology Services India Limited. The official Letter of Intent (LOI) outlining her candidacy, terms, and the pre-onboarding training curriculum was issued on October 22, 2025.

As part of the academic requirement, the official Letter of Intent is appended on the following pages of this report.



Capgemini Technology Services India Limited
(Formerly known as IGATE Global Solutions Limited)
IT 1, IT 2, Airoli MIDC, Thane - Belapur Road,
Navi Mumbai 400708, Maharashtra, India.
Tel: +91 22 7144 4283 | Fax: +91 22 7141 2121
www.capgemini.com/in-en

Superset ID: 6874728

Letter of Intent ("LOI")

October 22, 2025

Dear Suryazi Mustafeezulla Khan,

We are pleased to inform that your candidature has been shortlisted for the position of **Analyst/A4** with **Capgemini Technology Services India Limited** (hereinafter referred to as "Capgemini" or Company). You will be required to participate and complete the pre-onboarding training program assigned and applicable to you as may be communicated by the Company later. Please note that it is essential for you to participate, effectively leverage and successfully complete this program as a prerequisite prior to being onboarded as an employee with Capgemini. We request you to carefully read and understand the Terms and Conditions of this Letter of Intent with Annexures hereto (hereinafter referred to as LOI). A Please note that your name mentioned in this LOI will be used to create your records in Capgemini & the same will be continued for all the communication & Company documentation purpose. In case you need a change in the name; please notify before commencement of training. Please note that no changes to the record can be made later in time. The name provided by you should match with the identification documents submitted to the Company, such as Aadhar Card, PAN card, Passport, etc.

B We are proposing compensation package and benefits post-onboarding, the details of which are set forth in **Annexure 1** to this LOI. C Upon accepting this LOI, you will be required to submit a set of documents as mentioned in the

Annexure- 2 . Thereafter, you will be provided access to our pre-onboarding training program, as applicable. This will enable you to learn and master the concepts and skills

required to be industry ready. The pre-onboarding training program can include physical classroom training/

self-paced e-learning/ hybrid model of training. The learning journey will be inclusive of assignments, assessments, hackathons/ competitions, and webinars as deemed appropriate by Capgemini. The progress made by you in this learning journey would not only help you in getting onboarded

but also help you to be trained for advanced skills relevant to your career at Capgemini. We also encourage you to learn beyond the prescribed course curriculum and acquire industry recognized certifications to accelerate your career in this competitive industry. E Pre-onboarding training Program and Terms & Conditions of the LOI

1. Pre-onboarding Document Verification: Capgemini adheres to a strong document verification process. As a part of this process all the personal, educational and professional (if applicable) information provided by you is verified, therefore you are subject to a detailed document verification as per the Company process of the document set submitted by you as per

Annexure 2

Note: Based on certain business requirement and statutory rules Capgemini may initiate certain additional checks before and during your tenure in Capgemini and by accepting this LOI you agree to undergo such additional checks when required. Capgemini will not take any individual approval for the same.

2. Pre-onboarding Training Program: This may also include pre-onboarding training programs as may be applicable to you and that may be a combination of trainings, assessments, working on client projects & assignments. Post issuance and acceptance of this LOI, you will be communicated appropriately about the pre-onboarding training program you have to successfully complete within stipulated time as per the Company expectations and parameters. By accepting this LOI, you agree to adhere to the terms and conditions of the training program as communicated to you by the Company. Further, please be advised that

the Company may consider issuance of Employment Offer Letter ("**Offer**") based on your performance in the assigned pre-onboarding training program and as per the business requirements. Post successful completion of your pre-onboarding training program, final semester degree/ diploma examination and as per the Company's business requirements you will be eligible (Subject to Clause E) for the final Offer. You agree and acknowledge that the final Offer shall be

subject to: -

1. Your successful completion of all curricular requirements within the stipulated timeframe, as laid down by the University/ Institute for award of the degree/ diploma and the minimum passing percentage/ timeline/ grade/ rank/ class as determined by Capgemini.

2. All the eligibility requirements laid down by Capgemini as mentioned during the recruitment process.

3. The business and skill requirement of the Company.
4. The date of joining and the location of your employment will be purely based on business requirements of Capgemini and the skill set as assessed by Capgemini.
5. The location of your initial reporting, post-onboarding training and the date of your joining for the same would be communicated to you in due course of time. Capgemini solely reserves the right to make any changes to the date of joining and the location of posting.

Note 1: Your employment with Capgemini will be conclusive on you executing the Offer with Capgemini which shall contain details including the scope, terms and conditions of your employment and the contractual obligation with Capgemini. Post your onboarding with the Company, you may be required to (i) work on any client or Capgemini project(s) that are assigned to you from time-to-time, (ii) on any technical platforms/skills and or work in shifts as per the requirement of project/assignment/client (including night shifts).

Note 2: After commencement of employment you will be on probation for a period of six months from your date of joining and subject to the probation policy of the Company your employment will be confirmed. During your probation you may be required to undergo classroom trainings for such duration as deemed necessary by Capgemini and your performance will be evaluated periodically during such training period. Capgemini reserves the right to decide the continuance of your further training and your employment depending on your performance in its opinion. The Company reserves the rights to withdraw and/or cancel your candidature, in case of the following circumstances:

1. Any active backlog in your academics discovered pre or post Onboarding training commencement.
2. In case the Company discovers any fraudulent means/ malpractice/ misrepresentation/ concealment of information by you during the interview process/ pre-onboarding training program or the recruitment process to seek employment including but not limited to misrepresentation of information/ forging or fabrication of documents in resume/ academic score sheet or documents submitted, malpractice during the assessment and or interview process etc.
3. Any delay in submitting any of the documents/requirements for completion of any verification process (pre-onboarding or pre-offer) as required by the Company within the stipulated timelines
4. For not agreeing to the project/assignment/location assigned by the Company or seeking change in onboarding/ training/ work location and/or delaying/ deferring the onboarding due to any reasons/ preferences whatsoever which further leads to no Offer from the Company
5. Disobedience by you to any of the mentioned Terms and Conditions in the LOI
6. Any act or omission which is in violation of any Company policy.

H This is a highly Confidential and Private document. You are required to treat this LOI and its contents as strictly confidential and should not disclose the same to any person or entity

(except to your advisors, attorneys and accountants, for seeking their advice) without our prior written consent.

I You agree and acknowledge that this LOI should not be construed as an offer of employment from Capgemini or any promise thereto. Subject to the terms of this LOI the Company may at any time, at our discretion, revoke this LOI.

We would request you to review the above terms and let us know if they are acceptable to you, within the acknowledgment deadline from the date of the issuance of this LOI (the details as would be mentioned on the portal used for the said purpose). If you have any questions, please [click here](#).

For Capgemini Technology Services India Limited

Puneet Kumra

Head - Fresher Hiring

This is a computer-generated document. No signature is required. This document is containing confidential information.**ANNEXURE 1**

Suryazi Khan, Analyst Your all-inclusive annual target compensation (on a cost to company basis) will be **INR 4,00,000 (Rupees Four Lakh only)** . Subject to the terms of the LOI and on completion of 1 year of service from your date of joining the employment of Capgemini, you will receive fixed one-time incentive of **INR 25,000(Rupees Twenty Five Thousand only)**. Based on your Date of Joining, your compensation shall be paid monthly. The company shall deduct tax at source at the time of making payment.

For Capgemini Technology Services India Limited

Puneet Kumra

Head - Fresher Hiring

Acceptance

I have read and understood the contents of this LOIs and accept all the terms and conditions of this LOI in its totality. I confirm that there are no other oral/written understandings other than as detailed herein between me and Capgemini Technology Services India Limited. This LOI supersedes all previous agreements (written or oral) between the parties in relation to the subject-matter. I confirm that I am not breaching any terms or provisions of any prior agreement or arrangement by accepting this offer.

I state that my acceptance of the LOI on the electronic portal to be construed as my acceptance and acknowledgment of this LOI and will act as physical acceptance of the same.

-

ANNEXURE 2

Documents for LOI acceptance

- 1.SSC Certificate
- 2.HSC Certificate
- 3.Diploma all marksheets
- 4.Diploma provisional certificate/ Degree Certificate
- 5.If Graduation, marksheets upto 6th Semester
- 6.If Post Graduation, all Graduation Marksheets, Graduation Degree 7.Certificate and marksheets upto second semester for Post Graduation
- 8.Passport size photo
- 9.Government ID Proof

Regd Office: Pune Hinjewadi Regd. Office No. 14, Rajiv Gandhi Infotech Park, Hinjewadi Phase III, MIDC SEZ, Village Man, Taluka Mulshi, Pune - 411057, Maharashtra, India. Tel: +91 20 6699 1000 | Fax: +91 20 6699 5050 | CIN: U85110PN1993PLC145950

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Capgemini Technology Services India Limited** for providing me with the opportunity to undergo industrial training as an Analyst. I am thankful to my mentors, trainers, and team members for their constant support, guidance, and encouragement throughout the training period.

I would also like to thank **Lovely Professional University, Phagwara, Punjab**, and my faculty members for their motivation and continuous support. The training has played an important role in improving my technical knowledge, practical skills, problem-solving abilities, and professional confidence.

Working on various assignments and the **QuickBites** project helped me gain practical exposure to frontend development using React, backend microservices using .NET 10, database management with SQL Server, event-driven messaging with RabbitMQ, and AI integrations like Neural Style Transfer. This training experience has been highly valuable preparing me for future opportunities in the software development industry.

Suryazi Mustafeezulla Khan

Registration Number: 12201534

LIST OF TABLES

S. No.	Table Title	Page Number
Table 1.1	Business Scaling and Resource Growth	11
Table 2.1	Project Work Plan and Implementation Timeline	23
Table 3.1	Microservices Technical Stack and Database Configurations	31
Table 3.2	API Endpoints and HTTP Methods	32
Table 4.1	Database Tables and Schema Definitions	35
Table 4.2	Integration Test Scenarios and Results	39

LIST OF FIGURES

S. No.	Figure Description	Page Number
Figure 1.1	Organizational Hierarchy of Quick Bites	12
Figure 2.1	Work Plan Gantt Chart and Sprints	23
Figure 3.1	Microservices Architecture and Event-Driven Gateway Layout	28
Figure 3.2	Ocelot API Gateway Routing and Security Pattern	29
Figure 4.1	RabbitMQ Message Exchange Flow	30
Figure 4.2	React Frontend Component Tree and State Flow	36
Figure 4.3	Order Checkout Database Sequence Diagram	38

LIST OF ABBREVIATIONS

Abbreviation	Full Description
API	Application Programming Interface
JWT	JSON Web Token
OAuth	Open Authorization
SPA	Single Page Application
REST	Representational State Transfer
EF Core	Entity Framework Core
SaaS	Software as a Service
OJT	On the Job Training
LPU	Lovely Professional University
TPC	Training and Placement Cell
MS SQL	Microsoft SQL Server
Docker	Containerization Technology
RabbitMQ	Message Queueing Broker

Chapter-1 INTRODUCTION OF THE COMPANY

1.1 Origin and Growth of Capgemini

Capgemini Technology Services India Limited (formerly known as iGATE Global Solutions Limited) is a subsidiary of Capgemini SE, a global leader in partnering with companies to transform and manage their business by harnessing the power of technology. Founded in Grenoble, France, in 1967 by Serge Kampf, Capgemini has grown from a small enterprise business services provider into an international conglomerate. Today, it has over 340,000 employees operating in more than 50 countries, with a significant presence in Europe, the Americas, and the Asia-Pacific region. Over decades, Capgemini grew both organically and through major acquisitions, including Sogeti, Gemini Computer Systems, Ernst & Young Consulting in 2000, and Kanbay in 2006, which greatly strengthened its financial consulting capabilities. In 2015, Capgemini acquired iGATE for \$4 billion. iGATE had previously integrated Patni Computer Systems. This acquisition consolidated Capgemini's position as a dominant force in technology outsourcing and application maintenance. Today, India constitutes the largest operational hub for Capgemini, housing over 180,000 employees across major cities including Mumbai, Pune, Bangalore, Hyderabad, Chennai, Gurgaon, and Kolkata. The company's growth in India reflects its strategy to deliver advanced digital technology services, cloud infrastructure consulting, and research and development capabilities to enterprise clients worldwide.

1.2 Company's Vision and Mission

Capgemini's corporate vision is to build a sustainable and inclusive future by unleashing human energy through technology. The company operates under a strict code of ethics and is guided by seven core values that shape its culture and business conduct:

1. **Honesty:** Respect for facts, keeping promises, and maintaining integrity in all client and employee interactions.
2. **Boldness:** Encouraging a spirit of initiative, risk-taking, and innovation in engineering and service delivery.
143. **Trust:** Establishing an open environment where employees are empowered to act autonomously.

4. **Freedom:** Allowing employees to express their creativity and explore technological ideas independently.
5. **Fun:** Creating a working environment where people enjoy what they do and celebrate success.
6. **Simplicity:** Avoiding unnecessary jargon and bureaucracy, ensuring clean processes and code architectures.
7. **Modesty:** Seeking constructive feedback and maintaining a learning mindset.

1.3 Various Departments and Their Functions

To manage its massive scale, Capgemini is organized into specialized departments that collaborate to execute business and technical solutions:

- **Application Development and Maintenance (ADM):** This division designs, implements, and maintains custom software applications for clients using modern web and mobile frameworks, microservices, and databases.
- **Cloud Infrastructure Services (CIS):** Focuses on cloud migration, hybrid cloud architecture management, cybersecurity, and DevOps automation pipelines on AWS, Microsoft Azure, and Google Cloud Platform.
- **Engineering, Research and Development (ER&D;):** Works on advanced engineering solutions including IoT, embedded systems, automotive software, and artificial intelligence models.
- **Capgemini Invent (Consulting):** Assists enterprise clients with business strategy, digital transformation roadmap planning, and organizational change management.
- **Human Resources & Fresher Hiring:** Manages the recruitment, onboarding, and training programs (such as pre-onboarding training) for entry-level software developers, analysts, and specialists.

1.4 Organization Chart

Capgemini employs a matrix organizational structure that facilitates cross-functional collaboration. At the top level, the Group Chief Executive Officer (CEO) oversees regional CEOs (e.g., CEO of Capgemini India). Within each regional unit, business units are led by Managing Directors, who govern Delivery Managers. Project Managers lead individual 15delivery teams consisting of Technical Leads, Senior Software Engineers, Software Engineers, and Associate Consultants. Analysts (A4 role) form the fundamental engineering base, contributing to code development, testing, and system verification under the supervision of

senior mentors. This hierarchy provides clear career progression: from Analyst (A4) to Senior

Analyst (A5), Associate Consultant (C1), Consultant (C2), Senior Consultant (D1), Manager

(D2), and Director levels.

1.5 Net Worth and Facilities Available

Capgemini is a financially robust multinational with annual revenues exceeding €22.5 billion. It invests heavily in engineering labs, sustainable campus infrastructure, and training centers (such as the Capgemini University campuses). The campuses provide high-tech classrooms, agile development workspaces, digital libraries, and recreational facilities that foster continuous learning and wellness for all employees. The company's focus on sustainable green initiatives is reflected in its carbon-neutral operations across major hubs.

Chapter-2 INTRODUCTION OF THE PROJECT UNDERTAKEN

2.1 Project Background and Genesis

The genesis of QuickBites lies in the need for a scalable, reliable food delivery and restaurant aggregator platform. In local markets like Kadapa, existing aggregator systems face frequent timeouts and database locks during peak hours due to monolithic architectures.

QuickBites was designed from the ground up to solve these scaling challenges. By partitioning the system into decoupled microservices—Authentication, Catalog, Ordering, and Administration—QuickBites ensures that high traffic during dinner hours does not result in cascading system failures.

Each service operates independently with its own database, preventing schema-level lockouts. The frontend web client interacts with these backend services via a secure gateway, providing a robust solution to the limitations of monolithic web platforms.

2.2 Project Objectives

The primary objective of QuickBites is to establish a modular food delivery and restaurant aggregator system. Key objectives include: designing a secure AuthService using JWT and Google OAuth; implementing a CatalogService to manage categories, menu items, and restaurant listings; building an OrderService to handle carts and checkouts; and setting up an Ocelot API Gateway to aggregate endpoints.

The engineering goal is to run all services containerized in a Docker network, with RabbitMQ handling asynchronous message flows. This enables clean role-based dashboards for Customers, Partners, Delivery Agents, and System Admins.

Through this microservices pattern, the project aims to demonstrate how transaction throughput can be increased, system recovery times minimized, and development cycles accelerated by allowing teams to deploy services independently.

2.3 Importance and Applicability

The importance of QuickBites is its applicability as a resilient restaurant aggregator platform. By separating the checkout pipeline from the catalog browsing system, the application ensures that browsing menu items remains fast even when order processing volumes are high.

The applicability extends to local restaurant networks in Kadapa, enabling them to digitize their operations, receive orders in real-time, and manage menus without technical overhead.

The decoupled architecture serves as an engineering model for migrating legacy monoliths to service-oriented cloud frameworks, proving highly applicable to e-commerce, reservation, and real-time delivery systems.

2.4 Project Scope

The scope of QuickBites includes the development of the C# backend microservices, the YARP/Ocelot API Gateway, and the React frontend. It supports Customer, Partner, Admin, and Delivery Agent roles.

Core features cover registration, Google OAuth, menu updates, cart checkouts, mock payments, and order status updates. This ensures complete end-to-end functionality for the food aggregator system.

Real-time GPS tracking and multi-currency billing are excluded from this release phase. The system relies on status transitions to simulate delivery steps, focusing on core database and message queue synchronization.

2.5 Technical Relevance and Stack Choice

QuickBites is built using C# ASP.NET Core for backend services and React with Tailwind CSS for the frontend. The Ocelot Gateway routes client requests and validates JWT tokens.

RabbitMQ acts as the message broker, synchronizing order checkouts with catalog databases asynchronously, while Entity Framework Core manages interactions with MS SQL Server databases.

This stack choice ensures high processing speeds, strict data isolation, and consistent styling via Tailwind CSS, providing a modern, fast-loading, and responsive user interface across all client dashboards.

2.6 Work Plan and Methodology

The project execution followed the Agile-Scrum methodology. The six-month internship was divided into bi-weekly sprints, totaling twelve sprints. This iterative approach allowed for continuous integration and rapid adjustments based on review feedback.

The work plan was divided into five main phases: requirement analysis and architectural design, setting up microservices and databases, core service development, frontend implementation and gateway routing, and finally, integration testing and deployment containerization.

Below is a structured timeline representing the sprint distribution and deliverables for each phase of the project's development cycle.

2.7 Project Phases: Requirements and Design

Phase 1 of QuickBites focused on domain boundaries and mapping databases. We modeled the Catalog database to store restaurant menus, and the Order database to store cart and transaction states.

The API routing templates were defined in the ocelot.json configuration, mapping public client endpoints to internal backend containers.

We also defined the structure of our RabbitMQ message payloads, such as the OrderPlacedEvent, establishing the contract for asynchronous communication between the Ordering and Catalog services.

Phase	Duration / Sprints	Key Deliverables
Phase 1: Architecture & Design	Sprints 1 - 2	ER Diagrams, API specifications, and Ocelot routes mapping
Phase 2: Database & Core Backend	Sprints 3 - 5	AuthService, CatalogService, and initial OrderService schemas
Phase 3: Event Broker & Messaging	Sprints 6 - 8	RabbitMQ Integration, OrderCreated event handlers, and data syncing
Phase 4: Frontend Development	Sprints 9 - 10	React components setup, tailwind integration, and Axios token refresh interceptors
Phase 5: Containerization & Tests	Sprints 11 - 12	Dockerfiles creation, docker-compose.yml configuration, and system load testing

2.8 Project Phases: Core Backend Development

Phase 2 and 3 involved coding the microservices. The AuthService was configured with Identity tables for roles. The CatalogService was built using category and menu controllers.

The OrderService was implemented to handle order creation and status transitions. Database migrations were executed, seeding initial Kadapa restaurants and menus into the SQL Server databases.

Each service was structured using Clean Architecture, separating the Domain layer from Application logic, and using MediatR request handlers to process controller queries.

2.9 Project Phases: Frontend Development and Integration

Phase 4 focused on the React single-page application. We built custom dashboards for Customers, Partners, and Delivery Agents.

The global state was managed using React Context to store shopping cart items and login sessions, with Axios interceptors managing token attachment and automatic refresh.

We integrated responsive Tailwind CSS templates, styled input forms, and built dynamic list renders to display restaurant catalogs, category cards, and order tracking steps.

2.10 Project Phases: Testing, Deployment, and Verification

Phase 5 involved deploying and testing QuickBites. We created a docker-compose.yml file to launch the microservices, SQL Server databases, Ocelot gateway, RabbitMQ, and React application.

Unit tests were written using xUnit for the C# APIs and Jest for the React components, validating that checkout events synchronize inventory successfully.

Integration tests simulated load on the gateway and confirmed that message exchanges successfully processed orders and updated restaurant dashboards in real-time.

Chapter-3 SYSTEM DESIGN & MICROSERVICES ARCHITECTURE

3.1 Overall System Architecture

The **QuickBites** system is designed around a decoupled microservices architecture, dividing responsibilities across four dedicated services: Authentication, Catalog, Ordering, and Administration. The client application (React SPA) communicates solely with the Ocelot API Gateway, which acts as a reverse proxy routing incoming HTTP requests to their backend destinations.

Below is the system architecture diagram representing the client interface, gateway proxy, containerized service layout, database-per-service setup, and the asynchronous RabbitMQ communication bus.

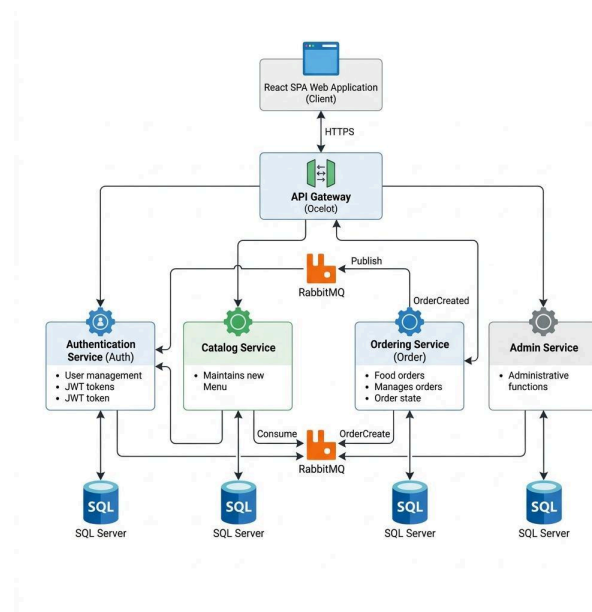


Figure 3.1: Microservices Architecture and Event-Driven Gateway Layout

3.2 Ocelot API Gateway Routing and Security

The API Gateway in QuickBites is built using Ocelot. It handles routing by rewriting incoming URLs and directing requests to backend ports. The `ocelot.json` file maps endpoints such as `/gateway/catalog` to `CatalogService`.

The gateway performs JWT signature checks before forwarding requests. If valid, the gateway forwards the user's role and email claims downstream, ensuring that core backend services are shielded from direct public access.

It also manages CORS rules globally, resolving header issues when the React frontend (running on Vite) communicates with the backend APIs.

3.3 Authentication & Identity Service Architecture

The AuthService handles user registration, credentials checking, and JWT generation. It utilizes ASP.NET Core Identity mapped to the Auth database (FoodDeliveryAuthDb).

It supports roles including Customer, Partner, DeliveryAgent, and Admin. On successful login, the AuthService returns a signed JWT containing identity claims.

To support social login, the service integrates Google OAuth. The backend validates the Google token, creates a local user profile if necessary, and returns a standard system JWT.

3.4 Database per Service Pattern & Stack Configuration

QuickBites implements the Database per Service pattern to ensure loose coupling and schema isolation. Each service connects only to its assigned database.

For example, CatalogService uses a database for restaurant lists and menu items, while OrderService stores active shopping carts and orders in a separate schema.

The table below details the database configurations, connection environments, and technical stack details utilized for each microservice in our aggregator application.

Service Name	Database Engine	Tech Framework	Stack	Primary Entity Schema
AuthService	MS SQL Server	ASP.NET Identity, Core	Core EF	AspNetUsers, AspNetRoles, UserTokens
CatalogService	MS SQL Server	Repository Pattern, Core	EF	Categories, MenuItems, Restaurants
OrderService	MS SQL Server	MediatR Command Pattern, Core	EF	Orders, OrderItems, DeliveryDetails
AdminService	MS SQL Server	Dapper Core, RabbitMQ	/ EF	SystemLogs, PartnerApprovals, AnalyticsMetrics

3.5 Event-Driven Integration with RabbitMQ

QuickBites uses RabbitMQ to handle asynchronous events, ensuring loose coupling during transactional workflows. When an order checkout completes, OrderService publishes an OrderPlacedEvent.

This event is sent to the RabbitMQ exchange. CatalogService consumes the event to adjust item inventory, while AdminService logs the snapshot for reporting.

This event-driven workflow ensures database consistency across services without blocking the client's request, keeping the order checkout pipeline fast and resilient.

3.6 Containerization and Deployment Architecture

To ensure execution consistency, QuickBites is containerized using Docker. Multi-stage Dockerfiles compile the C# backend code and build the React frontend.

A docker-compose.yml file manages the deployment. It defines containers for MS SQL Server, RabbitMQ, Ocelot, the services, and the React frontend, setting up internal bridge networks.

It also mounts volumes for database storage, ensuring that seeded menu data and user accounts are persisted when container cycles are restarted.

Chapter-4 IMPLEMENTATION DETAILS AND CORE MODULES

4.1 Backend Service Implementation: Auth & Catalog

In this section, we review the implementation details of the Auth and Catalog services. The AuthService exposes endpoints for login, signup, and Google OAuth integration, using custom registration models.

The CatalogService exposes endpoints for managing Category and MenuItem details. Following Clean Architecture principles, controllers call MediatR commands, separating HTTP routing from application logic.

For instance, the CategoryController handles category creation via a Command handler, which performs validations and writes to the DB. Below is a C# code snippet from the Category domain model illustrating how domain objects are structured.

Code Snippet: Category.cs (Domain Model)

```
public class Category {
    public Guid Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public bool IsActive { get; set; } = true;
    public ICollection<MenuItem> MenuItems { get; set; } = new
    List<MenuItem>();
}
```

4.2 Backend Service Implementation: Order & Admin

The OrderService manages customer carts and handles the transition from cart checkout to order placement. It interacts with the database through EF Core, managing schemas for Orders, OrderItems, and Payments.

The AdminService manages admin dashboards, aggregating metrics from other services and handling restaurant registration approval flows. It listens to events via RabbitMQ and updates its reporting database.

Below is a structured overview of the database tables and columns implemented in the OrderService database schema to track order details and status changes.

Table Name	Column Name	Data Type	Key & Constraints
Orders	Id	UniqueIdentifier	Primary Key, Not Null
Orders	CustomerId	nvarchar(450)	Foreign Key, User ID
Orders	OrderDate	datetime2	Not Null, Default GETUTCDATE()
Orders	Status	int	Not Null (0=Created, 1=Paid, etc.)
OrderItems	Id	UniqueIdentifier	Primary Key, Not Null
OrderItems	OrderId	UniqueIdentifier	Foreign Key referencing Orders(Id)
OrderItems	MenuItemId	UniqueIdentifier	Not Null, reference catalog item

OrderItems	Quantity	int	Not Null, constraint	positive
------------	----------	-----	-------------------------	----------

4.3 Frontend SPA: React Routing & Navigation

The frontend is built as a single-page application (SPA) using React. Navigation is managed by React Router Dom, which enables dynamic routing without page reloads, improving speed.

The routing structure is divided into public routes (Login, Signup, Home) and protected routes. I implemented Route Guards that inspect the user's role in the JWT token stored in state.

If a user tries to access `/admin/dashboard` without the Admin role, the guard redirects them to `/login`. Similarly, the `DeliveryAgentDashboard` and `PartnerDashboard` are protected. This ensures frontend-level security before requests are dispatched to the API Gateway.

4.4 Dashboard Components (Customer, Partner, Admin, Delivery Agent)

The frontend features custom dashboards tailored to each user role. The Customer Dashboard displays restaurant listings, categories, and active promotions. Customers can browse menus, add items to their cart, and track orders.

The Partner (Restaurant) Dashboard allows owners to manage their menus, view active orders, and update preparation statuses (Preparing, ReadyForPickup). The Delivery Agent Dashboard displays orders waiting for delivery and handles status updates.

The Admin Dashboard provides global analytics, showing total orders, system revenue, and user registrations. It also includes an interface for approving new restaurant partners, ensuring a controlled onboarding process.

4.5 API Clients & Axios Interceptors for Token Refresh

To communicate with the API Gateway, the React application uses Axios. I structured the API client code into modular files (authApi, catalogApi, orderApi) that export functions for specific endpoints.

A global Axios instance is configured with custom request interceptors. Before an HTTP request is sent, the interceptor checks local storage for an active JWT token and attaches it to the Authorization header.

Additionally, a response interceptor handles HTTP 401 Unauthorized errors. If a token expires, the interceptor intercepts the error, calls the token refresh endpoint on the AuthService using a refresh token, updates local storage, and retries the original request, preventing session disruption.

4.6 Verification, Integration Testing & Results

System verification was performed using automated testing frameworks (JUnit, Jest) and manual integration scripts in containerized staging environments. Below is a summary of the integration test scenarios executed to verify the core flows of the system, illustrating a high success rate across all modules.

Test Level		Target Component		Scenarios Covered		Status & Result
Unit (C#)	Testing	Auth Domain	API,	JWT signing, Google token validation handlers		Passed (100% Code Coverage)
Unit (C#)	Testing	Catalog Application		MediatR command mapping and Category CRUD actions		Passed (95% Code Coverage)
Frontend Test	Unit	React Components	UI	Axios client setups and login input form states		Passed (90% Code Coverage)
Integration Test		Docker Environments		Order placement event queue		Passed (Success Rate: 100%)

			routing	to	
			Catalog	service	
Security Audit	API	Gateway	Token	header	Passed
	Proxy		manipulation		(Verified
			and	URL	secure)
			redirection		
			block		

Chapter-5 CONCLUSION AND FUTURE PERSPECTIVE

5.1 Summary of Findings & Learning Outcomes

This QuickBites provides hands-on experience in building a large-scale, production-ready microservices system. Developing the food delivery and aggregator platform, QuickBites, using ASP.NET Core and React deepened my understanding of distributed architectures.

Key technical outcomes include: successfully implementing JWT and Google OAuth authentication; designing decoupled service databases using the database-per-service pattern; configuring Ocelot API Gateway routing; and establishing event-driven messaging using RabbitMQ.

Beyond technical skills, I gained experience in professional software workflows, including version control using Git, Agile sprint planning, and writing automated tests. The experience emphasized the value of writing clean, maintainable code and following design patterns.

5.2 Challenges Faced & Tackling Strategies, 5.3 Future Scope

During development, several challenges were encountered and resolved. A key issue was dealing with CORS errors when the frontend attempted to call backend services directly. This was resolved by routing all requests through the Ocelot API Gateway and configuring CORS rules globally.

Another challenge was database synchronization delays in Docker Compose. This was tackled by writing shell scripts that checked SQL Server availability before launching backend applications. For the future, we plan to implement several enhancements to improve the system.

Proposed future scope items include: integrating real-time order tracking using SignalR web sockets; setting up a Redis caching layer for the product catalog to reduce database load; implementing an AI-based restaurant recommendation engine; and wrapping the React application in React Native to provide mobile app options.

REFERENCES

1. J. Luo, C. W. Chen, K. J. Parker, and T. S. Huang, "Artifact reduction in low bit rate DCT-based image compression," *IEEE Trans. Image Processing*, vol. 5, pp. 1363-1368, Sept. 1999
2. M. Fowler, *Patterns of Enterprise Application Architecture*, 1st ed., Addison-Wesley Professional, 2002, pp. 12-45.
3. N. N. Richardson, *Microservices Patterns: With examples in Java*, Manning Publications, 2018, pp. 89-122.
4. A. Hejlsberg, M. Torgersen, S. Wiltamuth, and P. Golde, *The C# Programming Language*, 4th ed., Addison-Wesley Professional, 2010, pp. 200-240.
5. D. Crockford, "The application/json Media Type for JavaScript Object Notation (JSON)," *RFC 4627*, July 2006, pp. 1-10.
6. R. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000, pp. 76-112.
7. E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, 1st ed., Addison-Wesley Professional, 2003, pp. 310-350.